

Detection of Phishing in Mobile Instant Messaging using Natural Language Processing and Machine Learning

Suman Verma, Vanessa Ayala-Rivera

School of Computing

National College of Ireland

Dublin, Ireland

x21154996@student.ncirl.ie, vanessa.ayala-rivera@ncirl.ie

A. Omar Portillo-Dominguez

School of Business Technology, Retail, and Supply Chain

Technological University Dublin

Dublin, Ireland

omar.portillo@tudublin.ie

Abstract—Advancements in mobile technology makes it easier to communicate in real time, but at the cost of having a wider potential attack area for phishing. While there has been research in the field related to Email and SMS, Instant Messages lags behind. The widespread usage of instant messengers by individuals of all ages further motivates the addition of software security features in this context. This research aims to detect phishing in mobile instant messages by analysing the language of the message with the help of Natural Language Processing to detect keywords pointing towards phishing. We built the machine learning models using 3 different methods for feature extraction and 3 classification algorithms. Our tests showed that balancing the data with random oversampling increased the classifiers' performance, which were able to achieve an accuracy up to 99.2%.

Keywords—Instant Messaging, Social Engineering, Phishing, Natural Language Processing, Secure Software Engineering

I. INTRODUCTION

Phishing is a crime that involves tricking people into giving up their personal and financial information through social engineering and technology [1]. Due to the development of applications for all purposes, mobile phone usage has increased dramatically with technological advancement. The recent decade has seen an increased use of Mobile Instant messaging (IM) applications like WhatsApp, Telegram; and the same way these have been target of social engineers for phishing scam, but there has also been a relative low interest to study phishing susceptibility in these applications despite the risks involved. For instance, the authors of [2] reported that users of IM applications tend to click on links without hesitation and forward them to friends and groups, which could result in many individuals falling for the scam because the link comes from their known contacts. Considering the report of Anti-Phishing Working Group [1], the second quarter (Q2) of 2022 saw 1,097,811 phishing assaults in total, setting a record and ranking it as the worst quarter for phishing and even smishing and vishing incidents rose in Q2 2022, indicating a rise in mobile-based fraud. The most common vector of phishing is the malicious email sent by an impersonator, from employer, service provider or randomly targeted, all aiming to extract crucial information such as

credentials or account details. Phishing aimed at IM is more potent, as it could target anyone from the society, even from a vulnerable group, due to the availability of mobile phones to everyone including children.

Phishers innovate new methods and, with text messages, it is easy to manipulate the intention in words to befriend first, gain trust next, and then apply the phishing scam. At present, most of the available tools and research to detect phishing are mainly concentrated towards email and SMS, while their detection in messages remains mainly neglected [3].

Regarding mobile computing, the small screen of a mobile might make it difficult to observe all the necessary features in a short span of time and, thus, the victim falls prey to the lure of the attacker [4]. Furthermore, an IM conversation can be in real time and can be very much influential, as well as personalized towards the victim with respect to the psychological state of mind [5]. Therefore, while finding solutions for phishing attacks through IM, the analysis of text messages plays a crucial role. Nevertheless, human language is complex and can have varied interpretations depending upon the context of the situation while in a conversation.

Due to the above challenges, and with the aim of contributing to a more secure mobile software environment, we are presenting an approach through this research work using machine learning (ML) and natural language processing (NLP) to identify and label phishing messages in IM.

The above research problem motivates the following research question: "How to improve effectiveness of phishing identification in IM using NLP based on ML?" The goal of this work is to analyse mobile IM for terms that could entice users into disclosing private information, clicking on harmful links, downloading executable files, or engaging in prohibited behaviour that could result in phishing attacks.

Regarding the proposed approach, three techniques of vectorisation have been applied to extract the features for training three different classifiers. Specifically, the Logical Regression Classifier, the Gaussian Naive Bayes Classifier, and the Random Forest Classifier have been experimented with three techniques of vector representation namely Bag

of words (BOW), Term Frequency-Inverse Document Frequency (TFIDF), and Word2vec. Moreover, the SMS Phishing dataset from Mendeley data [6] was used for training and testing purposes.

The experimental results have been evaluated by comparing and analysing different standard ML metrics such as the accuracy of the classification and the recall percentage of phishing messages. Overall, the obtained results have shown that the classifiers performed well with an accuracy over 84% when trained and tested with document matrix using BOW. When the dataset was balanced with Random Over Sampling (ROS) and vectorized using TFIDF, Random Forest Classifier outperformed other models with accuracy for phishing detection of 99.2% and recall of 85%. Word2vec method of word embedding when used with Random Forest Model performed the classification with an accuracy of 95.2% with the dataset. Thus, the obtained results are promising and illustrate the benefits that might be gained by integrating a NLP and ML classifier, as a model, into a mobile IM application. This is because such approach might help in better detecting and blocking phishing messages for the benefit of the IM provider and public users in general.

The rest of the paper is structured as follows: The related work is discussed in section II. The research methodology is discussed in section III. Section IV describes the design specification of the proposed model to detect IM phishing. The implementation of the model is discussed in section V, whereas section VI presents its evaluation and discussion of the results. Finally, Section VII concludes the research paper with pointers discussed for future work in the area.

II. RELATED WORK

Detection of phishing in communication media, if left solely on the shoulders of human, could be flawed because of their fragile nature and other factors that typically consider human beings as the weak link, and susceptible to social engineering attacks. This may not always be the case, but this problem could be more appropriately fixed by training a system to detect phishing rather than relying on people. Thus, the inclusion of phishing detection models by the IM providers can be a great way to reduce the risk of phishing. Phishers attempt to act as reliable conversation partners on IM applications, then requesting personal and/or sensitive information to get access to the unaware victim's sensitive information, such as passwords and security codes for bank accounts. For instance, the authors of [7] pointed that IM can contain multiple elements other than text (e.g., images, files). Further, some of the IM applications support voice and video calls, thus offering an ideal environment for social engineers. As the name implies, IM enables real-time communication, enabling phishers to engage with targets and coerce them into disclosing personal information through scams. Given that criminal tactics and methods of operation are well known, as well as the fact that phishing incidents

are routinely made public, the issue of what has been done to stop this nuisance naturally emerges. Without a doubt, research is being done to determine how to counter this threat, and new technologies are being developed to strengthen our defences while every available option is being tested against it. The following sections briefly describe the main themes in the area.

A. Training to End Users

Various studies have assessed the efficacy of updating the system and users as one method to stop phishing at any level. It is usually advisable to offer training to end users and users at all levels when it comes to updating users. The authors of [8] suggested training for people that might provide them real-world experience with social engineering tactics than just warning about the dangers, workable solutions, and recommendations. The training must consider and eliminate the user's individual vulnerability for it to be effective. Meanwhile, the work presented in [9] pointed out that only 35% of users they researched were aware of threats against mobile devices such as phishing, vishing, and smishing and, thus, could be easy targets. They further remarked in their study that men were more susceptible to attack than women on the facts and figures of men being more technical savvy and thus found themselves comfortable in establishing trust in cyberspace easily. They proposed to advertise a dictionary of probable "intriguing" and "deceptive" terms used in phishing attempts for end users as a starting point to avoid falling for these social engineering attacks. Overall, these studies towards training to end users regarding phishing attacks highlight the need for a well-structured and practical approach that will not only highlight the damage, but at the same time equip them with the knowledge to identify and report it.

B. Content Analysis and URL Behaviour

The authors of [10] proposed an anomaly-based approach of URL and user behaviour to detect phishing in IM, which could contain a malware file, or a link to a malicious website. They approached the proposal by matching the URL with known malicious behaviour and developed a score model to predict the malicious URL in real time. Furthermore, the work of [11] proposed to detect phishing in email and messages with the presence of a malicious link in it. The research applied ML to detect malicious URL features like long length of URL, shortened URL, presence of @, extra dots, etc. Likewise, the work of [12] too presented an approach to detect smishing in SMS through content analysis and URL behaviour termed as *Smishing Detector*. The proposed model consisted of four modules such as a SMS analyser, URL filter, source code analyser, and APK downloader. NLP and ML models were used to detect malicious intent and keyword in content of messages that contained an email or phone number. The result showed

accuracy of 96.29% using a Naive Bayes classifier, and the model provided far better security than others. The above works do reinforce the idea of content analysis of email, SMS, and IM only on the presence of malicious link, which could not be always true and helpful in the current mobile landscape.

C. Data Mining Approaches

The authors of [13] proposed the detection of phishing in IM using a data mining technique of association rule mining and information retrieval technique. They demonstrated an association rule mining method (i.e., an a priori algorithm) to identify fraudulent phishing and suspicious communications delivered through IM between multiple clients. The IM targeted used for the research had phishing attempts, where attackers attempt to learn the password and other security-related information through queries. Furthermore, the work of [14] refined the above-mentioned work for phishing identification in IM using a Classification Based Association (CBA) and domain ontology. They further stated that phishing identification could be ascertained by determining the context of the phishing terms identified. These, when combined with the retrieved domain, can produce more intriguing phishing rules that will produce instant, dynamic phishing alerts for the victim with a high performance, or maximum true positives percentage based on CBA rules. Together these studies have researched to detect phishing in IM applications and presented promising results.

D. NLP and MP Approaches

The authors of [13] suggested using NLP to analyse conversations in IM to extract any query or command as likely themes to detect social engineering. To determine if they were malicious or not, they were also compared with topic backlists. Likewise, the authors of [15] suggested a similar strategy based on the semantic analysis of the text of IM using NLP. Working with a Spanish dataset, the authors of [16] developed this strategy further by extracting more characteristics from the text for analysis with an accuracy of over 80% utilizing Neural Networks, Support Vector Machine (SVM), and Random Forests. This research focused on one-way and two-way communication, and the user reports that contain plain text are utilized to compile the data on social engineering attacks from various sources such as email, SMS, and chats. Meanwhile, the authors of [17] proposed a model to detect social engineering attacks occurring during a conversation using NLP and deep learning. It suggests a social engineering assault detection model that processes dialogue context using attention-based Bi-Long short-term memory (LSTM) and fuses user characteristics and text characteristics together using a Residual Networks (ResNet) model. The viability and effectiveness of the proposed model are shown by examining the approach objects' properties in terms of the rationale and application of algorithm selection.

Moreover, the work presented in [18] compared several NLP techniques namely (TFIDF, Word2Vec, and Bidirectional Encoder Representations from Transformers -BERT-) and ML approach combinations for the detection of phishing emails to determine the best combination based on performance. The findings of this research work indicated that the Word2Vec methodology performed best with ML phishing email detection strategy based on the text content of the mail. Particularly, the Word2Vec along with Random Forest and Word2Vec along with Logical Regression combinations produced the best outcomes in the evaluated balanced and imbalanced datasets, respectively. Moreover, the authors of [19] presented a review of various ML algorithms for detection of phishing in electronic messaging service, with an average accuracy over 90%. SVM and Random Forest outperformed others with a 99.87% accuracy. The dataset size, source, and train-test size varied across the research. Similarly, the authors of [20] presented a study of different ML models using different pre-processing steps. Random Forest and Gradient boost outperformed other models with all pre-processing steps. It also advised the use of Word2vec as a feature extraction step. Together, these research efforts have explored the application of NLP in phishing detection, primarily focused on the text contents of emails or SMS messages, and showed promising results. This encourages us to use NLP and ML for phishing detection on mobile IM.

III. RESEARCH METHODOLOGY

As previously discussed, our research goal was to build a classification model to detect phishing in mobile IM application using the NLP and ML. For the above-mentioned purpose, we used the research methodology visually depicted in Fig. 1. Its steps are described in more detail next:

- *Dataset Selection:* Based on our literature review, we selected the open-source dataset of SMS phishing from Mendeley data [12].
- *Cleaning and Data pre-processing:* The chosen dataset was cleaned and pre-processed in order to be utilized by our ML models.
- *Vectorisation:* Representation of processed text into machine-readable form was performed by three different methods of vectorisation (i.e., BOW, TFIDF and Word2vec).
- *Building the classification model:* Three supervised ML classifiers were trained and evaluated on the dataset (i.e., Random Forest, Logical Regression, and Gaussian Naive Bayes).
- *Cross-validation and Evaluation:* The different classifiers had been cross-validated, and parameter tuned for evaluation and selection of model.

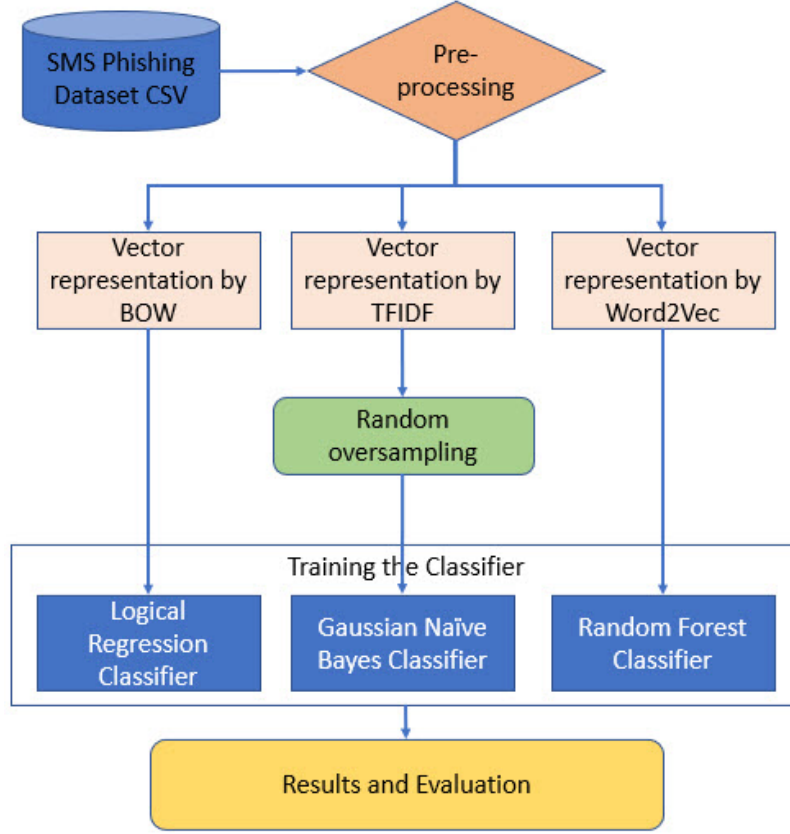


Figure 1. Flowchart of NLP on SMS Phishing dataset using ML Models.

Table I
ORIGINAL DATASET LABEL.

Label	Number	Percentage
Legitimate (Ham)	4844	81.8
Spam	489	8.1
Smishing	638	10.6

A. Dataset

The "SMS phishing dataset for ML and pattern recognition" used for the project was open source and available on Mendeley data [12]. The dataset had a collection of labelled text messages used in SMS phishing research. It contained 5971 texts messages labelled as Legitimate/Ham (4844), Spam (489), and Smishing (638), as detailed in Table I.

As per our research objective, we needed to classify the messages as phishing or no phishing, thus we regrouped the labels. This involved renaming the Smishing label as *Phishing*, as well as renaming both Spam and Ham labels as *No Phishing*. The total number of renamed labels is shown in Table II.

The dataset obtained for this research work had five different columns of properties: respectively as label, text, URL, email, and phone, with a total of 5971 messages

Table II
DATASET LABEL FOR THE RESEARCH.

Label	Number	Percentage
No Phishing (Ham + Spam)	5333	89.3
Phishing (Smishing)	638	10.6

	LABEL	TEXT	URL	EMAIL	PHONE
0	ham	Your opinion about me? 1. Over 2. Jada 3. Kusr...	No	No	No
1	ham	What's up? Do you want me to come online? If y...	No	No	No
2	ham	So u workin overtime nigpun?	No	No	No
3	ham	Also sir, i sent you an email about how to log...	No	No	No
4	Smishing	Please Stay At Home. To encourage the notion o...	No	No	No

Figure 2. Raw Dataset.

(i.e., rows). URL, email and phone columns are Boolean values representing the presence or absence of these their respective type of information within the text of the messages. Fig. 2 presents a sample of the raw dataset.

B. Data Pre-processing

The dataset to be worked upon need to be processed as the messages may contain information irrelevant for the

analysis which, if present, might add unnecessary noise to the process. Pre-processing of data presented better features for analysis, avoid consumption of unnecessary resources, and delay in the result. Different steps of pre-processing were performed as follows:

- *Replacing the mobile number, email, and URL pattern:* This information was replaced in the text with their respective constant keywords (i.e., "mobile number", "email", and "hyperlink") for the purpose of cleaning and a better understanding of the dataset.
- *Converting text to lower case:* The computer could identify the same word in lower and capital letters as different words, leading to confusion during the training. Thus, the text column of the dataset was converted to lower case.
- *Removing special characters:* Special characters present in the text (e.g., ? &, , @) could also account for noise, so they had been removed during pre-processing.
- *Removing numeric characters:* Any numeric value presents in the text had been removed, as they would have added to the noise only.
- *Words Tokenisation:* Tokenisation is the division of text into words and sentences. The unstructured data could be converted into structured one by using tokenisation. Thus, we used word tokeniser to identify the luring, compelling words that might lead to a phishing attack through messages.
- *Removing short words:* The text data contained short words (e.g., a, an, spl), which did not contribute to understand the text purpose. Thus, they had been removed.
- *Stop word removal:* Different stop words (e.g., the, an, a) had also been removed from the text.

C. Vectorisation

Vectorisation turns human-written material into comprehensible numerical representations, or machine-readable forms. In our research, we had used three different methods of vectorisation to represent the pre-processed data for ML. The aim of using three different methods is to compare and analyse the best one to achieve the detection of phishing in IM correctly. In addition to the vectorisation techniques used for representation of text of messages, we did use hot encoding for vector representation of labels in the dataset.

Encoding the categorial column: To our project, the LABEL of dataset had been replaced with PHISHING and regrouped further where, Smishing = Phishing; and Ham + Spam = no phishing. The categorical dependent column, that is Phishing or Legitimate, had been encoded for the purpose of classification as Phishing = 1 and no phishing = 0.

Data transformation/vectorisation: ML models needed data to be processed in the form of numbers, as the computer could not understand and interpret the textual information unless presented in a meaningful numeric representation. The following three techniques had been used for the

creation of the representation of textual data of messages in the form of numbers (i.e., a machine-readable form):

- *BOW:* It is the simplest model for representation of words in vector form. Here the frequency of repeated words is considered and does not count the order of words in the document. Length of vectors keeps on increasing with each new word being added in the dataset and the matrix looks sparse 0s being added if any word is absent in that document. There is no order of words in the text being followed in the matrix, which could lead to hindrance of meaning. The Python Count Vectoriser library was used to apply BOW. N-grams can be used to implement the Count Vectoriser. Utilizing an unigram and bigram matrix, the feature was retrieved for this research. The Bigram matrix consisted of two consecutive words from a document, while the Unigram matrix featured one word. Fig. 3 shows some sample rows of the vectorized dataset using BOW.
- *TFIDF:* It is a typical technique used for converting text data into a vector format. It aids in comprehending the word's significance throughout the corpus of documents. Here, two things are calculated, that is, the term frequency (which is the number of times a word appears in the document); and the Inverse Document Frequency (which means the log of the number of documents divided by the number of documents containing that word). Hence, the more frequent words used in the document were weighted low in terms of vector representation, but less frequent and important words had high IDF value as vector representation [18]. An example vector representation is presented in Fig. 4.
- *Word2vec:* It is a technique used to represent the words in a dense vector form in space, where the words related in contexts have similar vectors. This is an advantage of using Word2vec in detection of phishing, as attackers keep modifying the language to avoid the direct bait of phishing. With the help of this tool, it is possible to turn words into vectors, calculate their distances, and construct an analogy between the words. Word2vec can identify the semantic connections between the words in a sentence [21]. Word2Vec builds word vectors, which are distributed numerical representations of word features. They may include words that indicate the context of the specific vocabulary words that are present individually. Through the generated vectors, word embeddings eventually assist in forming the relationship of a word with another word having a similar meaning. The Gensim library [22] was used to create the word embedding of the dataset. Some arrays of transformed vectors are presented in Fig. 5.

D. Classification Model

The text data from IM needed to be classified into phishing and no phishing, as per the aim of this research.

	PHISHING	abiola	able	accept	access	account	account block	account statement	across	activate	...	yest	yesterday	yijue	yoga	yogasana	youd	youll	youre	yc aro
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0
4	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0

Figure 3. Vector representation using BOW.

_____	aaniye	aaoooooight	aapka	aapki	aathilove	aathiwhere	abbey	abdomen	abeg	...	zahers	zealand	zebra	zero	zhong	zindgi	zogtorius	zoom	:
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Figure 4. Vector representation using TFIDF.

```
array([-0.1552314 , 0.33461034, 0.04795472, -0.00757872, 0.11274458,
       -0.6175206 , 0.12564117, 0.6670057 , -0.13389097, -0.25220987,
       -0.1420534 , -0.5027224 , 0.06805111, 0.2017314 , 0.09599011,
       -0.31923822, 0.08968898, -0.46911538, 0.02925646, -0.6596575 ,
       0.22181797, 0.19733022, 0.23375732, -0.19647591, -0.11062763,
       -0.00727809, -0.287007 , -0.2007781 , -0.3892724 , 0.03578618,
       0.30387613, -0.06130878, 0.1462432 , -0.18069567, -0.24450806,
       0.34971833, -0.08045572, -0.27334073, -0.1844069 , -0.67064744,
       0.17774294, -0.36700633, -0.03762876, -0.10462865, 0.28455752,
       -0.12016756, -0.38750193, -0.05744408, 0.2776591 , 0.20143454,
       0.11758444, -0.29793194, -0.03829952, -0.09961917, -0.19771987,
       0.13517173, 0.19590399, -0.11133744, -0.3447317 , 0.06185888,
       -0.03011188, -0.01208152, 0.04391169, -0.0768849 , -0.4406703 ,
       0.2530809 , 0.14245111, 0.29409084, -0.46476603, 0.4412023 ,
       -0.18316036, 0.05886695, 0.2640255 , -0.02624813, 0.36945093,
       0.15636659, 0.04105894, 0.06874377, -0.3766654 , 0.06198136,
       -0.10261787, -0.02126191, -0.31779405, 0.5867131 , -0.09593919,
       -0.05602019, 0.07156706, 0.39123255, 0.4231187 , 0.13543543,
       0.5023025 , 0.17776865, -0.05215285, 0.04370752, 0.5601385 ,
       0.3150802 , 0.19991513, -0.3116203 , 0.13258702, -0.09984916],
      dtype=float32)
```

Figure 5. Vector representation using Word2vec.

We employed three different classifiers to compare the findings and determine which was better for the task. The three supervised ML models used were: Random Forest, Gaussian Naive Bayes, and Logistic Regression. They were selected because of their performance and frequent use by the research community. The first stage in creating a classification model was to train the chosen model using training data and then test the model with test data. A train test split of 0.33 was used to break the vectorised data for the purpose of training the models and evaluating it.

Logistic Regression Classifier: Logistic regression classifier is based on the principle of linear regression, but since

the output is classifying the data into binary, hence fits the classification model. Here, Regression is subjected to a logistic function to determine the likelihood that it belongs to either class. It compares the log of the likelihood that an event will occur to the log of the likelihood that it will not. In the end, it categorizes the variable according to which class has the highest likelihood [20].

Naive Bayes Classifier: A straightforward probability technique based on the idea that each feature of the model is independent of the others. In the context of the phishing filter, we assume that each word in the message is distinct from every other word, and we tally the words without taking context into account [19].

Random Forest Classifier: One of the most popular ML techniques for classification and regression. It combines the results of many decision trees' calculations to produce a single outcome [19]. It is well-liked because it is straightforward yet efficient. Random forests are based on the idea that while each tree might be rather good at predicting, it will almost certainly overfit on some data. They typically work well without a lot of parameters changing, are quite powerful, and do not require data scalability.

E. Cross Validation and Evaluation

The ML models could be biased if the train and test data is not utilised properly, so we did imply the stratified K-fold cross validation. Stratified K-fold cross validation helped to create a number of K-fold in a manner that both classification label will be randomly distributed, but proportionally in all the folds of split used for training and testing of the classifier. We did 10-fold cross validation for our research. Further parameters of individual classifiers were tuned to improve the specificity and efficiency of models.

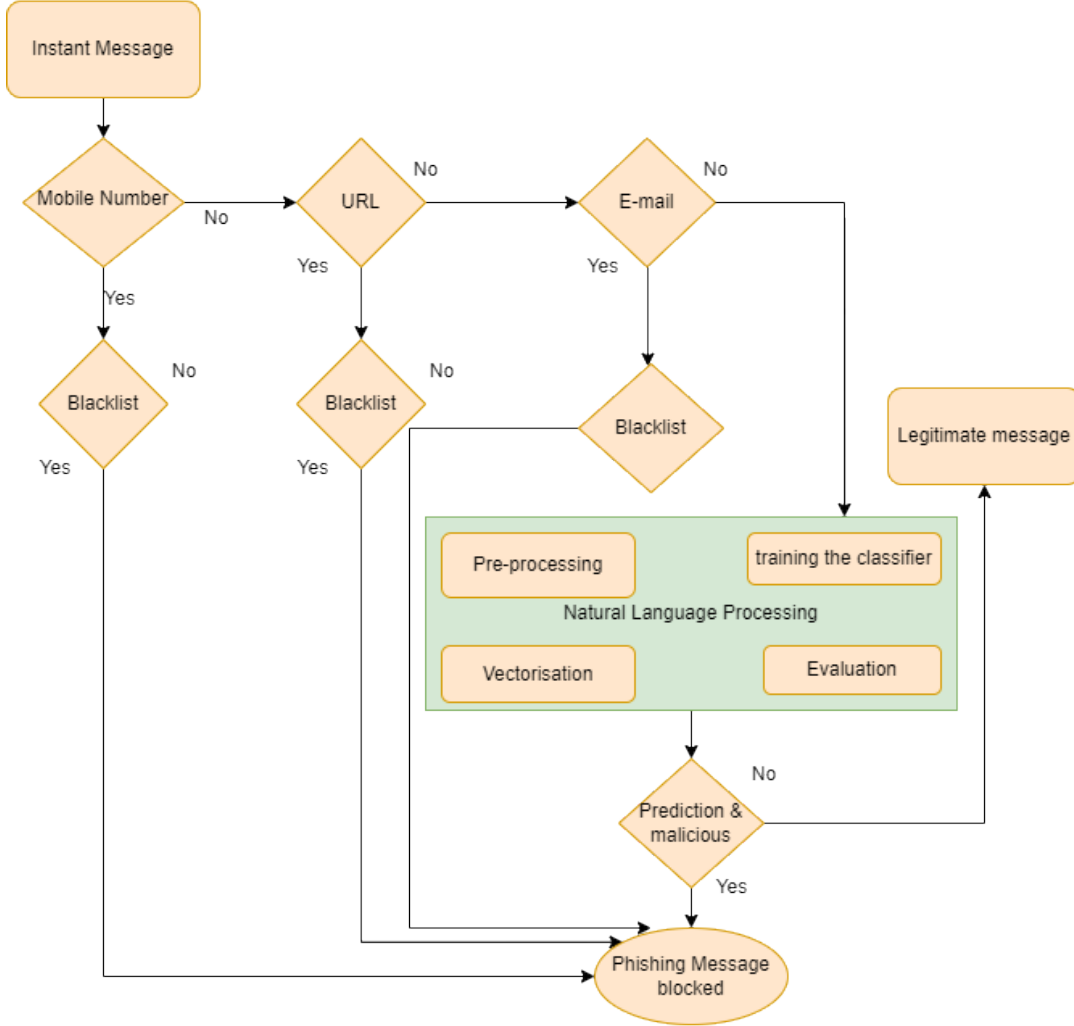


Figure 6. Possible system design to leverage our proposed model for phishing detection in Mobile IM.

IV. DESIGN SPECIFICATION

Fig. 6 shows a possible way to incorporate our proposed model of Phishing detector into a Mobile IM. Next, we describe the main identified components:

Malicious mobile number analyser: The text messages could be sent from a deny-list mobile number, or could contain a deny-list number for further communication. In this situation, the mobile number will be checked within the deny list and, if it matches, the model will flag the message as phishing and block it.

Malicious URL link analyser: The text messages might contain a malicious link, thus, if any link is present, it would need to be checked in a deny list. If present, the model would flag the message as phishing and blocked it.

Malicious email analyser: The text messages could contain a deny-list email for further communication. If the email exists in the deny list, the model will also flag it as phishing and block it.

IM analyser using NLP and ML: As attackers might invest time in building the trust through earlier conversation, before initiating a phishing attack, it is important to comprehend the context of words in messages to better predict phishing and be able to classify and stop phishing messages before they reach consumers. As previously described, our research work has focused on addressing this aspect of IM vulnerability through the usage of NLP, where the text would be analysed for the presence of malicious keywords potentially used in phishing attacks. In our research, we have also experimented with different vectorisation methods and different classifiers to extract the best features to classify the IM as phishing and no phishing.

V. IMPLEMENTATION

To demonstrate the potential benefits of this research work, the previously described approach was implemented in Python. Next, we described the developed prototype:

Environment: On Windows 11, we installed the open-source Anaconda Distribution and utilized Jupyter Notebook version 6.4.8 as the development and learning environment for our project and ML model creation.

Dataset: We used the Pandas library to read the dataset and perform data analysis.

Pre-processing: Steps of pre-processing has been explained previously. To perform this step, we used the following libraries: NumPy (used for mathematical operations), and NLTK (Natural Language Toolkit, framework to build NLP programs with inbuilt libraries for tokenisation, lemmatisation, stemming, stop word removal).

ML: The scikit-learn library was used to implement the models, as well as to conduct a variety of ML-related tasks like feature extraction using CountVectoriser, TfidfVectorizer, train_test_split. Evaluation metrics (e.g., confusion_matrix, classification_report) were also used. The gensim library was also used for topic modelling, document indexing, and similarity retrieval.

Data balancing: The dataset utilised for the analysis was imbalanced with a 9:1 ratio of legitimate and phishing message, so we did implied ROS on our X train and Y train set of data. The imblearn library was used for this purpose.

k-fold cross validation: ML algorithms needed to be tested for accuracy through cross-validation to avoid over-fitting on dataset. Here, we employed a 10-fold cross validation by analysing the model to be cross validated 10 times by splitting the dataset into 10 groups and training and testing on these groups. scikit-learn library was used for this purpose.

Data visualisation: The matplotlib library had been used for visualising the data for evaluation.

Parameter tuning: This helped to get the best and consistent result for the analysis (e.g., n_estimator with cross_val_score was used to tune the Random Forest model).

VI. EVALUATION

The evaluation of the different models' performance was based on the following standard ML metrics:

- **Accuracy:** The proportion of accurate predictions to all other predictions.
- **Precision:** The fraction of positive predictions that were accurate.
- **Recall:** The percentage of actual positives that were mistakenly detected.
- **F1 Score:** The harmonic mean of the precision and recall.
- **Receiver Operating Characteristics (ROC):** It displays the Total Positive Rate (TPR) and the False Positive (RateFPR).
- **Area under the ROC Curve (AUC):** It measures the area under the ROC curve.

Table III
PERFORMANCE METRICS OF CLASSIFIERS FOR PHISHING MESSAGES WITH BOW.

Classifier	Accuracy	Precision	Recall	F 1 score
Logistic Regression	0.97	0.91	0.79	0.84
Gaussian Naïve Bayes	0.84	0.38	0.88	0.54
Random Forest	0.97	0.9	0.79	0.85

Table IV
PERFORMANCE METRICS OF CLASSIFIERS FOR PHISHING MESSAGES WITH TFIDF.

Classifier	Accuracy	Precision	Recall	F 1 score
Logistic Regression	0.97	0.8	0.91	0.85
Gaussian Naïve Bayes	0.87	0.45	0.77	0.57
Random Forest	0.97	0.89	0.84	0.87

Furthermore, we trained the in-scope ML classifiers with 3 different feature extraction methods: BOW, TFIDF, and Word2vec. Their experimental setups are described next:

A. Experimental Setup 1: BOW

A cross-validation strategy was used. It involved the number of K-folds increased with the use of stratified K-fold cross validation, ensuring that both categorisation labels would be distributed randomly; but proportionately throughout all folds of the split. All the models were trained and cross validated with a Stratified K-fold having $k = 10$. Table III presents a summary of its results. They are discussed in the next section.

B. Experiment Setup 2: TFIDF

A cross-validation strategy, similar to the previous experiment, was used. It involved the number of K-folds increased with the use of stratified K-fold cross validation, ensuring that both categorisation labels would be distributed randomly, but proportionately throughout all folds of the split. All the models were trained and cross validated with a Stratified K-fold having $k = 10$. Furthermore, we vectorised the dataset using TFIDF as a method for vectorisation, as it gives more weightage to less used terms in the document, which might be a better way to select features for training and testing in our case. To balance the dataset, we used ROS (Random Over Sampling) on the X train and Y train sets of data. The document matrix obtained is further used to train and test different classifier to get better understanding of analysis. The results are presented in Table IV, and also discussed in the next section.

C. Experiment Setup 3: Word2vec

Finally, we used the Word2vec word embedding method (from the Gensim package), as a method for vectorisation. Word2vec is a vector representation of the dataset that considers the context of the words used in a message. As a result, Word2vec assigns similar vector scores to words with similar context. This might benefit the training of the

Table V
PERFORMANCE METRICS OF RANDOM FOREST CLASSIFIER WITH
WORD2VEC.

Accuracy	Precision	Recall	F1 score
0.95	0.87	0.66	0.75

classifier. Similar to the other experiments, we used a 10-fold stratified cross-validation. Finally, as Random Forest Classifier responded well and, overall, outperformed the other two models in the previous experiments, we exclusively used the Random Forest Classifier in this final experiment. Table V presents the results obtained.

D. Experimental Results - Discussion

Three different combination of NLP techniques (BOW, TFIDF, and Word2vec) and ML models (Logical regression, Gaussian Naive Bayes, and Random Forest) have been experimented and evaluated in this research work. The effectiveness of the classification models was assessed, based on the recall of phishing messages, as well as the accuracy of predicting the right labels of legitimate and no phishing messages in IM.

In general, all the ML algorithms performed well, with an accuracy of phishing detection over 84% in case of BOW vectorisation (as shown in Table III). When using TFIDF as vectorisation method, it weighted the less frequent words more and, thus, proved to be a better feature extraction technique for training the classifiers. The train dataset was further balanced to avoid overfitting. The models trained with TFIDF showed a high performance, where Random Forest outperformed the others by achieving a 99.2% accuracy (as shown in Table IV). Moreover, when using Word2vec word embedding, only the Random Forest classifier was trained and tested due to its consistent performance in previous experiments. Here, the Random Forest presented an accuracy of 95.2% upon cross validation. Hereby, we evaluated the performance of three supervised models with three different techniques of vectorisation, as represented in Table VI. The table presents the matrix of evaluation to provide the most effective model and approach. Finally, Fig. 7 presents the bar chart for model evaluation based on Accuracy and Recall (in %) of the different models using different vectorisation techniques.

VII. CONCLUSIONS AND FUTURE WORK

This research presents a phishing detection and classification approach in instant messaging using NLP and ML algorithms. By contrasting several vectorisation techniques and ML models for the detection of phishing IM, we developed a method to build such detection approach. The BOW, TFIDF, and Word2vec methods of vectorisation using NLP were utilised to create a document matrix from the messages. Each document matrix created was then trained and tested on the Logical Regression, Gaussian Naive Bayes,

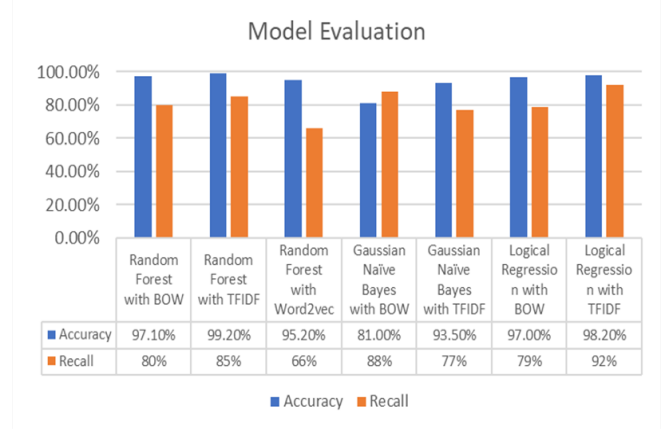


Figure 7. Model evaluation based on accuracy and recall (in %) of different models using different vectorisation techniques.

and Random Forest classifiers for the purpose of classifying messages into phishing or no phishing. When the dataset was balanced with ROS and vectorized using TFIDF, Random Forest Classifier outperformed the other models achieving an accuracy for phishing detection of 99.2%, and recall of 85%, for phishing messages.

Based on our work in the area, we consider that there is plenty of room for further improvements in the same. For instance, the lack of a well-defined dataset for IM can be a hinderance towards our goal of using NLP to detect phishing in IM using contextual relationships between words and sentences. This could be remedied by developing a specific dataset and having society participate by updating it frequently. As a future work, we will focus on developing datasets more specific for IM phishing. Additionally, our proposed model can be combined with others to better detect likely phishing messages from fraudulent mobile numbers, containing malicious email addresses, malicious URLs, and/or executable files on a large scale. Thus, we will explore ways to implement such capabilities. Finally, we will also explore the integration of NLP for classification with other ML techniques in order to further enhance the appropriate detection of IM phishing.

REFERENCES

- [1] APWG, "Phishing activity trends report, 2nd quarter 2022," <https://apwg.org/trendsreports/>, Accessed: 2023-07-31.
- [2] R. Ahmad and S. Terzis, "Understanding phishing in mobile instant messaging: a study into user behaviour toward shared links," in *Human Aspects of Information Security and Assurance: 16th IFIP WG 11.12 International Symposium, HAISA 2022, Mytilene, Lesbos, Greece, July 6–8, 2022, Proceedings*. Springer, 2022, pp. 197–206.
- [3] F. Salahdine and N. Kaabouch, "Social engineering attacks: a survey," *Future Internet*, vol. 11, no. 4, p. 89, 2019.

Table VI
PERFORMANCE METRICS OF CLASSIFIERS.

Model	Random Forest		Logical Regression		Gaussian Naïve Bayes	
Vectorisation	Accuracy	Recall	Accuracy	Recall	Accuracy	Recall
TFIDF with ROS data	0.992	85	0.982	92	0.935	77
Word2vec	0.952	66	NA	NA	NA	NA
BoW	0.971	80	0.97	79	0.81	88

- [4] A. K. Jain, N. Debnath, and A. K. Jain, "Apuml: an efficient approach to detect mobile phishing webpages using machine learning," *Wireless Personal Communications*, vol. 125, no. 4, pp. 3227–3248, 2022.
- [5] A. Stone, "Natural-language processing for intrusion detection," *Computer*, vol. 40, no. 12, pp. 103–105, 2007.
- [6] S. Mishra and D. Soni, "Sms phishing dataset for machine learning and pattern recognition," in *Proceedings of the 14th International Conference on Soft Computing and Pattern Recognition (SoCPaR 2022)*. Springer, 2023, pp. 597–604.
- [7] R. Alabdan, "Phishing attacks survey: types, vectors, and technical approaches," *Future internet*, vol. 12, no. 10, p. 168, 2020.
- [8] J. W. Scheeres, "Establishing the human firewall: reducing an individual's vulnerability to social engineering attacks," Air Force Institute of Technology Wright-Patterson, Tech. Rep., 2008.
- [9] E. O. Yeboah-Boateng and P. M. Amanor, "Phishing, smishing & vishing: an assessment of threats against mobile devices," *Journal of Emerging Trends in Computing and Information Sciences*, vol. 5, no. 4, pp. 297–307, 2014.
- [10] D. Guan, C.-M. Chen, and J.-B. Lin, "Anomaly based malicious url detection in instant messaging," in *Proceedings of the joint workshop on information security (JWIS)*, vol. 43, 2009.
- [11] C. Singh *et al.*, "Phishing website detection based on machine learning: a survey," in *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*. IEEE, 2020, pp. 398–404.
- [12] S. Mishra and D. Soni, "Smishing detector: a security model to detect smishing through sms content analysis and url behavior analysis," *Future Generation Computer Systems*, vol. 108, pp. 803–815, 2020.
- [13] Y. Sawa, R. Bhakta, I. G. Harris, and C. Hadnagy, "Detection of social engineering attacks through natural language processing of conversations," in *2016 IEEE Tenth International Conference on Semantic Computing (ICSC)*. IEEE, 2016, pp. 262–265.
- [14] M. V. Dass, M. A. Rasheed, and M. M. Ali, "Classification of lung cancer subtypes by data mining technique," in *Proceedings of the 2014 international conference on control, instrumentation, energy and communication (CIEC)*. IEEE, 2014, pp. 558–562.
- [15] T. Peng, I. Harris, and Y. Sawa, "Detecting phishing attacks using natural language processing and machine learning," in *2018 IEEE 12th international conference on semantic computing (icsc)*. IEEE, 2018, pp. 300–301.
- [16] J. C. Lopez and J. E. Camargo, "Social engineering detection using natural language processing and machine learning," in *2022 5th International Conference on Information and Computer Technologies (ICICT)*. IEEE, 2022, pp. 177–181.
- [17] Y. Lan, "Chat-oriented social engineering attack detection using attention-based bi-lstm and cnn," in *2021 2nd International Conference on Computing and Data Science (CDS)*. IEEE, 2021, pp. 483–487.
- [18] P. Bountakas, K. Koutroumpouchos, and C. Xenakis, "A comparison of natural language processing and machine learning methods for phishing email detection," in *Proceedings of the 16th International Conference on Availability, Reliability and Security*, 2021, pp. 1–12.
- [19] A. Kovač, I. Duner, and S. Seljan, "An overview of machine learning algorithms for detecting phishing attacks on electronic messaging services," in *2022 45th Jubilee International Convention on Information, Communication and Electronic Technology (MIPRO)*. IEEE, 2022, pp. 954–961.
- [20] N. Sharma, "A methodological study of sms spam classification using machine learning algorithms," in *2022 2nd International Conference on Intelligent Technologies (CONIT)*. IEEE, 2022, pp. 1–5.
- [21] S. Al-Saqqa and A. Awajan, "The use of word2vec model in sentiment analysis: a survey," in *Proceedings of the 2019 international conference on artificial intelligence, robotics and control*, 2019, pp. 39–43.
- [22] RARE technologies Ltd., "Gensim library," <https://radimrehurek.com/gensim/>, Accessed: 2023-07-31.